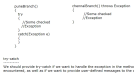
***Why compiler takes very much care regarding the checked Exception ?**

As we know Checked Exception are very common exception as in case of checked exception "handling is compulsory" ,The code should be terminated normally because checked Exception depends upon other resources outside.

IOException (we are depending upon System.exit(0) OR File)
FileNotFoundException(files are depending upon the file)
ClassNotFoundException (class related problem)
CloneNotFoundException (class related problem)
SQLException (SQL related or database related problem)
InterruptedException (Thread related problem)
DataFormatException (Data related problem)
EOFException(We are depending upon the file)

When to provide try-catch or declare the method as throws for Checked Exception :



try-catch

We should provide try-catch if we want to handle the exception in the method where checked exception is occurred, as well as if we want to provide user defined messages to the client.

throws

Throws keyword describes that the method might throw an Exception, It also might not (Exception always recursive to caller method) but if it does, it will be handled by the caller method. If multiple Exceptions it supports OR what type of Exception it might throw which will be handled by JVM OR caller method.

Rule – It is always better to use try catch as we can provide appropriate user defined messages to our client.

Exception propagation [Propagation of Exception Object from Callee to Caller method]

```
public static void main(String[] args)
{
    System.out.println("Main method started");
    m1();
    System.out.println("Main method ended");
}

public static void m1();
{
    System.out.println("M1 method started");
    m2();
    System.out.println("M1 method ended");
}

public static void m2();
{
    System.out.println("M2 method started");
    m3();
    System.out.println("M2 method ended");
}
```


AE is generated at m3 stack frame but not any of the caller method (m2 & main) handle it, so the JVM OR JVM will terminate all the Stack Frame and JVM default handler will print the exception details as well as terminate the program abnormally.

Whenever we call a method and if the callee method contains any kind of exception (checked OR Unchecked) and if callee method doesn't contain any kind of exception handling mechanism (try-catch OR throws) then JVM will terminate the callee method and propagate the exception object to the caller method for handling purpose.

If the caller method also does not contain any kind of exception handling mechanism then JVM will terminate the method in the stack frame handling the exception and the remaining methods will not be executed even if we handle the exception in another caller method like main.

If any of the the caller method does not contains any exception handling mechanism (try-catch) then exception will reach to the main() method, default exception handler which will provide the exception message and terminates the program ABNORMALLY.

1.Program on Exception Propagation using Unchecked

```
package com.rsd.exception.propagation;

public class ExceptionPropagationUnchecked
{
    public static void main(String[] args)
    {
        System.out.println("Main method started");
        m1();
        System.out.println("Main method ended");
    }

    public static void m1()
    {
        System.out.println("M1 method started");
        try
        {
            m2();
        }
        catch(ArithmeticException e)
        {
            System.out.println(" HANDLED IN M1 METHOD");
        }
        System.out.println("M1 method ended");
    }

    public static void m2()
    {
        System.out.println("M2 method started");
        m3();
        System.out.println("M2 method ended");
    }
}
```

2.Program on checked exception

```
package com.rsd.exception.propagation;

class ChennaiBranch
{
    static
    {
        System.out.println("Static block of ChennaiBranch class");
    }
}

public class ExceptionPropagationCheckedDemo
{
    public static void main(String[] args)
    {
        System.out.println("Main method started");
        m1();
        m2();
        m3();
        catch(ClassNotFoundException e)
        {
            System.out.println(" HANDLED IN MAIN METHOD");
        }
        System.out.println("Main method ended");
    }

    public static void m1() throws ClassNotFoundException
    {
        System.out.println("M1 method started");
        m2();
        System.out.println("M1 method ended");
    }

    public static void m2() throws ClassNotFoundException
    {
        System.out.println("M2 method started");
        Class.forName("com.rsd.exception.propagation.ChennaiBranch");
        System.out.println("M2 method ended");
    }

}
```